Experiment Log

This file is an ongoing experiment log for model/environment changes.

After each improvement, add one new entry with the exact training command and final metrics.

Standard Evaluation Setup

- Environment: `MinesweeperEnv(rows=5, cols=5, num_mines=3, seed=42)`
- DQN training: `python3 train_dqn.py --num-episodes 5000 --progress-every 500`
- Final evaluation: 1000 games with DQN epsilon set to `0.0`
- Baselines in report:
  - `RandomAgent` (1000 eval games)
  - `QLearningAgent` (trained, then evaluated with epsilon `0.0`)

Current Default Workflow

- Main model to improve: `CNN_DEEP DQN`
- Current difficulty target: `8x8`, `10` mines
- Main seed for fast iteration: `55`
- Default focused evaluation command:
  - ```
    python3 compare_dqn_models.py --rows 8 --cols 8 --num-mines 10 --models cnn_deep
    --replay-modes uniform --reward-modes progress --seeds 55 --num-episodes 10000
    --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999
    --save-model-path models/best_cnn_deep_8x8_10m_progress_seed55.pt
    ```
- Previous best saved 5x5 checkpoint:
  - `models/best_cnn_deep_5x5_seed55.pt`
- Current 8x8 checkpoint target:
  - `models/best_cnn_deep_8x8_10m_progress_seed55.pt`
- Multi-seed comparisons should only be repeated when a change looks promising enough to justify a broader validation pass.

Metrics Legend

- Win rate: fraction of games won
- Average reward: average total reward per episode
- Average steps: average number of steps per episode

Entry 001 - Initial DQN Baseline

- Date: 2026-04-24
    ```

- Change: Initial DQN implementation (`agents/dqn_agent.py`, `train_dqn.py`)
- Hypothesis: DQN should beat RandomAgent and tabular QLearningAgent on this 5x5 setting.
- Result: See the comparison plot below.
- Notes: This result came from the first full DQN run after implementation.

Entry 002 - Input Scaling for Revealed Cells

- Date: 2026-04-24
- Change: Normalize DQN inputs in `_flatten_observation`:
  - unknown stays `-1.0`
  - revealed `0..8` is scaled to `0.0..1.0` by dividing by `8.0`
- Hypothesis: More consistent input scale may improve stability/learning quality.
- Command:
  - `python3 train_dqn.py --num-episodes 5000 --progress-every 500`
- Progress snapshots:
  - Episode 500: win `11.80%`, avg_reward `-3.406`, avg_steps `3.936`
  - Episode 2500: win `20.60%`, avg_reward `-0.030`, avg_steps `4.584`
  - Episode 5000: win `28.00%`, avg_reward `2.338`, avg_steps `4.658`
- Final Result: See the comparison plot below.
- Outcome: **Degraded vs Entry 001** (clear in the plot). This change was reverted in the next experiment.

Entry 003 - 2-Channel State Input (Hidden Mask + Revealed Values)

- Date: 2026-04-24
- Change:
  - Reverted single-channel normalization-only input.
  - Switched DQN state to 2 channels flattened and concatenated:
    - Channel 1: hidden mask (`1.0` hidden, `0.0` revealed)
    - Channel 2: revealed normalized values (`0..8` mapped to `0.0..1.0`, hidden=`0.0`)
  - Updated target-mask logic to read valid actions from hidden-mask channel.
- Hypothesis: Separating "visibility" signal from "number value" signal should be more learnable than mixing both in one scalar.
- Command:
  - `python3 train_dqn.py --num-episodes 5000 --progress-every 500 --loss-plot-path dqn_loss_curve_entry003.png`
- Progress snapshots:
  - Episode 500: win `9.00%`, avg_reward `-4.288`, avg_steps `3.922`
  - Episode 2500: win `28.20%`, avg_reward `2.518`, avg_steps `4.776`

- Episode 5000: win 42.40%, avg_reward 7.040, avg_steps 4.896
- Final Result: See the comparison plot below.
- Outcome: Strong improvement. Better than Entry 001 and much better than Entry 002. Keep this as the new best configuration.

Result Plot

Referenced plot: plots/encoding_comparison.svg

Entry 004 - Reliable 5-Seed Evaluation + CNN DQN Variant

- Date: 2026-04-25
- Change:
  - Added reliable multi-seed evaluation for DQN.
  - Added CNN-based DQN variant while keeping the same 2-channel input:
    - Channel 1: hidden mask
    - Channel 2: revealed normalized values
  - Preserved replay buffer, target network, epsilon-greedy, and valid-action masking.
- Hypothesis: CNN should use board structure better than MLP and improve win rate/reward.
- Command:
  - `python3 compare_dqn_models.py --num-episodes 5000 --eval-games 1000 --seeds 11 22 33 44 55 --progress-every 1000`
- Per-seed results (MLP):
  - Seed 11: win 44.50%, reward 7.829, steps 5.034
  - Seed 22: win 45.80%, reward 8.310, steps 5.112
  - Seed 33: win 46.70%, reward 8.122, steps 4.645
  - Seed 44: win 46.50%, reward 8.155, steps 4.740
  - Seed 55: win 47.60%, reward 8.731, steps 4.975
- Per-seed results (CNN):
  - Seed 11: win 59.60%, reward 13.176, steps 5.700
  - Seed 22: win 56.80%, reward 12.064, steps 5.456
  - Seed 33: win 58.90%, reward 12.703, steps 5.444
  - Seed 44: win 54.30%, reward 11.329, steps 5.496
  - Seed 55: win 60.20%, reward 13.262, steps 5.600
- Mean ± std across seeds:
  - MLP DQN: win 46.22% ± 1.03%, reward 8.229 ± 0.295, steps 4.901 ± 0.178
  - CNN DQN: win 57.96% ± 2.16%, reward 12.507 ± 0.727, steps 5.539 ± 0.097
- Outcome: CNN clearly outperformed MLP on win rate and reward across all 5 seeds. New best model is CNN DQN.

Entry 005 - Huber Loss (SmoothL1Loss)

- Date: 2026-04-25
- Change:
  - Replaced DQN loss from `MSELoss` to `SmoothL1Loss` (Huber loss).
  - Kept the same 2-channel input, replay buffer, target network, epsilon schedule, and model architectures.
- Hypothesis: Huber loss might make training more robust to large TD-error spikes and improve stability.
- Command:
  - `python3 compare_dqn_models.py --num-episodes 5000 --eval-games 1000 --seeds 11 22 33 44 55 --progress-every 1000`
- Per-seed results (MLP with Huber):
  - Seed 11: win `34.90%`, reward `4.594`, steps `4.775`
  - Seed 22: win `35.40%`, reward `4.727`, steps `4.753`
  - Seed 33: win `27.90%`, reward `1.935`, steps `4.286`
  - Seed 44: win `33.10%`, reward `3.858`, steps `4.597`
  - Seed 55: win `33.30%`, reward `4.050`, steps `4.727`
- Per-seed results (CNN with Huber):
  - Seed 11: win `50.10%`, reward `9.703`, steps `5.172`
  - Seed 22: win `53.40%`, reward `10.987`, steps `5.433`
  - Seed 33: win `49.50%`, reward `9.584`, steps `5.239`
  - Seed 44: win `53.10%`, reward `10.603`, steps `5.142`
  - Seed 55: win `49.40%`, reward `9.503`, steps `5.189`
- Mean ± std across seeds:
  - MLP DQN: win `32.92% ± 2.66%`, reward `3.833 ± 1.003`, steps `4.628 ± 0.182`
  - CNN DQN: win `51.10% ± 1.77%`, reward `10.076 ± 0.603`, steps `5.235 ± 0.104`
- Outcome:
  - Huber loss degraded both architectures relative to Entry 004.
  - Previous MSE-based CNN remained better: `57.96%` win rate vs `51.10%` with Huber.
  - Recommendation: revert Huber loss and keep MSE for now.
  - Status: reverted after this experiment.

Entry 006 - CNN Seed 55: `epsilon_min=0.05` vs `0.1`

- Date: 2026-04-25
- Change:
  - Compared the current best setup with only one change:
    - `epsilon_min=0.05`
    - `epsilon_min=0.1`

- Focused only on the current main target:
  - `CNN DQN`
  - seed `55`
- Hypothesis: A slightly higher exploration floor (`0.1`) might help the CNN keep discovering useful states later in training.
- Commands:
  - `python3 compare_dqn_models.py --models cnn --seeds 55 --num-episodes 5000 --eval-games 1000 --progress-every 1000 --epsilon-min 0.05`
  - `python3 compare_dqn_models.py --models cnn --seeds 55 --num-episodes 5000 --eval-games 1000 --progress-every 1000 --epsilon-min 0.1`
- Result:
  - `epsilon_min=0.05`: win `60.20%`, reward `13.262`, steps `5.600`
  - `epsilon_min=0.1`: win `59.10%`, reward `12.990`, steps `5.669`
- Outcome:
  - `0.05` stayed slightly better than `0.1` on the current focused evaluation.
  - Keep `epsilon_min=0.05` as the default for now.

Entry 007 - Prioritized Experience Replay

- Date: 2026-04-25
- Change:
  - Added prioritized replay as an optional replay mode.
  - Kept the current CNN Double DQN architecture unchanged.
  - Kept the environment unchanged.
  - Kept the epsilon schedule unchanged.
  - Priorities are updated from absolute TD error plus `priority_epsilon`.
  - Importance-sampling weights are applied to the per-sample MSE loss.
- Default PER command:
  - `python3 compare_dqn_models.py --models cnn --replay-modes uniform prioritized --seeds 55 --num-episodes 5000 --eval-games 1000 --progress-every 1000`
- Default PER result:
  - Uniform replay: win `64.00%`, reward `14.405`, steps `5.565`
  - Prioritized replay (`alpha=0.6`, `beta_start=0.4`): win `57.30%`, reward `12.425`, steps `5.662`
- Tuned PER command:
  - `python3 compare_dqn_models.py --models cnn --replay-modes prioritized --seeds 55 --num-episodes 5000 --eval-games 1000 --progress-every 1000 --alpha 0.4 --beta-start 0.6 --beta-end 1.0 --priority-epsilon 1e-5`
- Tuned PER result:

- Prioritized replay (`alpha=0.4`, `beta_start=0.6`): win `61.60%`, reward `13.757`, steps `5.661`
- Outcome:
  - Gentler PER improved over default PER.
  - Uniform replay still remains the best current seed-55 result: win `64.00%`, reward `14.405`.
  - Recommendation: keep PER available as an experiment option, but keep uniform replay as the default.

Entry 008 - Reward Shaping: Newly Revealed Cells

- Date: 2026-04-26
- Change:
  - Kept existing penalties unchanged:
    - invalid action: `-5`
    - already revealed action: `-2`
    - mine/loss: `-10`
  - Added `reward_mode=progress`:
    - safe move reward: `0.5 * newly_revealed`
    - win reward: safe move reward plus `25`
  - Kept architecture, replay method, epsilon schedule, and environment rules unchanged.
- Command:
  - `python3 compare_dqn_models.py --models cnn --replay-modes uniform --reward-modes classic progress --seeds 55 --num-episodes 10000 --eval-games 1000 --progress-every 500`
- Result:
  - Classic reward: win `58.70%`, reward `12.907`, steps `5.710`
  - Progress reward: win `62.40%`, reward `20.297`, steps `5.459`
- Outcome:
  - On focused seed `55`, progress reward improved win rate and average steps.
  - Average reward is not directly comparable because the reward scale changed.
  - This is promising, but still needs a 3-seed validation before treating it as the new default.

Entry 009 - Lower Exploration Floor for CNN_DEEP

- Date: 2026-04-26
- Change:
  - Kept the current best focused architecture and setup:
    - `CNN_DEEP DQN`
    - uniform replay
    - classic reward

- seed 55
- Changed only the exploration schedule:
- `epsilon_min=0.001`
- `epsilon_decay=0.999`
- Hypothesis:
- Minesweeper is highly sensitive to random bad clicks, so lowering the late-training random-action floor may improve final policy quality.
- Slower decay keeps exploration high for longer before settling near-greedy behavior.
- Command:
- `python3 compare_dqn_models.py --models cnn_deep --replay-modes uniform --reward-modes classic --seeds 55 --num-episodes 10000 --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999`
- Progress snapshots:
- Episode 500: win `10.80%`, reward `-3.670`, epsilon `0.606`
- Episode 3000: win `46.60%`, reward `8.172`, epsilon `0.050`
- Episode 6000: win `65.80%`, reward `15.068`, epsilon `0.002`
- Episode 8500: win `71.00%`, reward `17.118`, epsilon `0.001`
- Episode 10000: win `69.20%`, reward `15.972`, epsilon `0.001`
- Final Result:
- CNN_DEEP DQN: win `69.20%`, reward `16.085`, steps `5.633`
- Comparison to previous best focused classic-reward result:
- Previous best: win `64.30%`, reward `14.425`, steps `5.492`
- New result: win `69.20%`, reward `16.085`, steps `5.633`
- Outcome:
- Lowering `epsilon_min` from the earlier `0.05` floor to `0.001` improved the focused seed-55 result.
- This is now the best known focused configuration.
- Because this was only one seed, the next validation step should be a 3-seed run before treating it as robust.

Entry 010 - Save Best 5x5 CNN_DEEP Checkpoint for UI Playback

- Date: 2026-04-26
- Change:
- Added DQN checkpoint save/load support.
- Saved the current best focused 5x5 model weights to:
- `models/best_cnn_deep_5x5_seed55.pt`
- Added `Saved Best DQN` to the UI agent selector.
- UI can now load the saved checkpoint with `epsilon=0.0` and play without retraining.

- Command:
- `python3 compare_dqn_models.py --models cnn_deep --replay-modes uniform --reward-modes classic --seeds 55 --num-episodes 10000 --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999 --save-model-path models/best_cnn_deep_5x5_seed55.pt`
- Final Result:
- CNN_DEEP DQN: win `69.20%`, reward `16.085`, steps `5.633`
- Checkpoint verification:
- Loaded checkpoint successfully as `cnn_deep`, board `5x5`, `epsilon=0.0`
- Outcome:
- The UI no longer needs to retrain the best model before playback.
- Use `Saved Best DQN` in `python3 ui_minesweeper.py` to watch the saved model play.

Entry 011 - Move Target Difficulty to 10x10 With 15 Mines

- Date: 2026-04-27
- Change:
- Selected the next board difficulty:
- rows: `10`
- cols: `10`
- mines: `15`
- Added board-size arguments to experiment scripts so `5x5` is no longer hard-coded:
- `compare_dqn_models.py`
- `train_dqn.py`
- `train_q_learning.py`
- `evaluate_random_agent.py`
- Updated the UI board to `10x10`, `15` mines.
- Updated the UI saved-checkpoint target to:
- `models/best_cnn_deep_10x10_15m_seed55.pt`
- Hypothesis:
- `10x10, 15 mines` is a meaningful next difficulty jump while keeping mine density reasonable at `15%`.
- The existing `CNN_DEEP` architecture can still run because it already supports variable rows/cols.
- Training command for first 10x10 run:
- `python3 compare_dqn_models.py --rows 10 --cols 10 --num-mines 15 --models cnn_deep --replay-modes uniform --reward-modes classic --seeds 55 --num-episodes 10000 --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999 --save-model-path models/best_cnn_deep_10x10_15m_seed55.pt`
- Baseline command:

- `python3 evaluate_random_agent.py --rows 10 --cols 10 --num-mines 15 --num-games 1000`
- Result:
- Classic reward partial run through episode `7500` showed near-zero wins but improving average reward:
  - Episode 500: win `0.00%`, reward `-6.232`
  - Episode 3000: win `0.00%`, reward `-4.008`
  - Episode 6000: win `0.60%`, reward `-2.734`
  - Episode 7500: win `0.20%`, reward `-2.172`
- Outcome:
  - Project is ready to train and evaluate `10x10` DQN.
  - The old `5x5` checkpoint should not be used on `10x10`, because output actions change from `25` cells to `100` cells.
  - Classic reward appears too sparse/weak for the larger board, so the next controlled run should use progress reward.

Entry 012 - Use Progress Reward for 10x10 Target

- Date: 2026-04-27
- Change:
  - Kept the `10x10`, `15` mines target.
  - Switched the UI DQN training/playback environment to `reward_mode=progress`.
  - Switched the UI checkpoint path to:
    - `models/best_cnn_deep_10x10_15m_progress_seed55.pt`
  - Added `--reward-mode` to baseline scripts so random/Q-learning comparisons can use the same reward mode.
- Hypothesis:
  - The classic reward gives too little learning signal on `10x10`.
  - Progress reward should help by rewarding the number of newly revealed cells, especially flood-fill openings.
- Training command:
  - ```
    python3 compare_dqn_models.py --rows 10 --cols 10 --num-mines 15 --models cnn_deep
      --replay-modes uniform --reward-modes progress --seeds 55 --num-episodes 10000
      --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999
      --save-model-path models/best_cnn_deep_10x10_15m_progress_seed55.pt
    ```
- Baseline command:
  - ```
    python3 evaluate_random_agent.py --rows 10 --cols 10 --num-mines 15 --num-games 1000
      --reward-mode progress
    ```
- Result:
  - Not run yet.
- Outcome:
  - This is the next recommended 10x10 experiment.
  - Compare against classic mainly by win rate and average steps; average reward is not directly comparable because the reward scale changed.

Entry 013 - Step Back to 8x8 With 10 Mines

- Date: 2026-04-27
- Change:
  - Moved the active target from `10x10`, `15` mines to:
    - rows: `8`
    - cols: `8`
    - mines: `10`
  - Kept the current promising setup:
    - `CNN_DEEP DQN`
    - uniform replay
    - progress reward
    - seed `55`
    - `epsilon_min=0.001`
    - `epsilon_decay=0.999`
  - Updated the UI board/checkpoint path to:
    - `models/best_cnn_deep_8x8_10m_progress_seed55.pt`
- Hypothesis:
  - `10x10`, `15` mines is a large jump from the successful `5x5` setup.
  - `8x8`, `10` mines keeps a similar mine density (`15.6%`) but reduces the action space from `100` to `64`.
  - This should be a more realistic next difficulty target before returning to `10x10`.
- Training command:
  - ```
    python3 compare_dqn_models.py --rows 8 --cols 8 --num-mines 10 --models cnn_deep
      --replay-modes uniform --reward-modes progress --seeds 55 --num-episodes 10000
      --eval-games 1000 --progress-every 500 --epsilon-min 0.001 --epsilon-decay 0.999
      --save-model-path models/best_cnn_deep_8x8_10m_progress_seed55.pt
    ```
- Baseline command:
  - ```
    python3 evaluate_random_agent.py --rows 8 --cols 8 --num-mines 10 --num-games 1000
      --reward-mode progress
    ```
- Result:
  - Not run yet.
- Outcome:
  - This is the new recommended intermediate-difficulty experiment.

Entry 014 - Curriculum Scaling + Optional Frontier Channel

- Date: 2026-04-28
- Change:

- Added `compare_curriculum_dqn.py`.
- Added curriculum training support:
  - train first on `5x5`
  - transfer CNN weights to `8x8`
  - transfer CNN weights to `10x10`
- Replay memory is reset between stages because stored states have board-specific shapes.
- Added per-stage configuration for:
  - board rows/cols
  - mine count
  - episodes
  - `epsilon_min`
  - `epsilon_decay`
  - replay `memory_size`
- Added optional frontier input channel:
  - `1.0` for hidden cells adjacent to at least one revealed numbered cell
  - `0.0` otherwise
- Kept the CNN layer pattern unchanged. The only network input change is optional 2 channels vs 3 channels.
- Hypothesis:
  - Direct `10x10` training is too large a jump from `5x5`.
  - Curriculum should transfer useful local Minesweeper patterns before training on the larger board.
  - The frontier channel may help larger boards by explicitly highlighting candidate cells near known information.
- Full comparison command:
  - `python3 compare_curriculum_dqn.py --modes direct curriculum curriculum_frontier --seeds 55 --eval-games 1000 --progress-every 500 --reward-mode classic`
- Smoke-test command:
  - `python3 compare_curriculum_dqn.py --stages 5x5x3:5:0.001:0.999:2000 8x8x10:5:0.05:0.9995:3000 10x10x15:5:0.10:0.9997:4000 --modes direct curriculum curriculum_frontier --seeds 55 --eval-games 5 --progress-every 0 --reward-mode classic`
- Smoke-test result:
  - Direct `10x10`: win `0.00%`, reward `-5.200`, steps `5.800`
  - Curriculum: win `0.00%`, reward `-5.200`, steps `5.800`
  - Curriculum + frontier: win `0.00%`, reward `-8.200`, steps `2.800`
- Reduced controlled comparison command:
  - `python3 compare_curriculum_dqn.py --stages 5x5x3:100:0.001:0.999:5000 8x8x10:100:0.05:0.9995:10000 10x10x15:100:0.10:0.9997:20000 --modes direct curriculum curriculum_frontier --seeds 55 --eval-games 100 --progress-every 100 --reward-mode classic`

- Reduced controlled comparison result:
  - Direct `10x10`: win `0.00%`, reward `-5.330`, steps `5.670`
  - Curriculum: win `0.00%`, reward `-5.910`, steps `5.090`
  - Curriculum + frontier: win `0.00%`, reward `-6.010`, steps `4.990`
- Outcome:
  - Smoke test only verifies that all three modes run end-to-end.
  - The reduced controlled comparison is still too short to evaluate final 10x10 learning quality.
  - At this small budget, direct `10x10` had the best average reward, while curriculum variants took slightly fewer steps before terminal states.
  - A real comparison still needs the full command above or a longer reduced run.

Entry 015 - Initialize 8x8 Curriculum From Saved Best 5x5 Checkpoint

- Date: 2026-04-28
- Change:
  - Added `--initial-checkpoint` to `compare_curriculum_dqn.py`.
  - Used the saved mature 5x5 model:
    - `models/best_cnn_deep_5x5_seed55.pt`
  - Transferred CNN weights into a new 8x8 agent, with a fresh replay buffer.
  - For frontier-channel runs, compatible CNN weights are copied and the extra frontier input channel keeps its random initialization.
- Hypothesis:
  - The previous curriculum run was undertrained because the 5x5 stage only reached about `21.60%` win rate.
  - Starting from the mature saved 5x5 model should give a more meaningful transfer test to 8x8.
- Command:
  - ```
    python3 compare_curriculum_dqn.py --stages 8x8x10:1000:0.05:0.9995:50000 --modes direct curriculum curriculum_frontier --initial-checkpoint models/best_cnn_deep_5x5_seed55.pt --seeds 55 --eval-games 500 --progress-every 250 --reward-mode classic
    ```
- Result:
  - Direct 8x8: win `0.20%`, reward `-5.064`, steps `5.874`
  - 5x5 checkpoint -> 8x8 curriculum: win `2.60%`, reward `-4.442`, steps `5.752`
  - 5x5 checkpoint -> 8x8 curriculum + frontier: win `2.40%`, reward `-3.978`, steps `6.278`
- Outcome:
  - This is the first evidence that transfer from the mature 5x5 checkpoint helps on 8x8.
  - Curriculum improved win rate by `+2.40` percentage points over direct 8x8 in this focused seed-55 run.
  - Frontier did not improve win rate over normal checkpoint curriculum, but it had the best average reward and longest survival.
    ```

- This should be validated with longer 8x8 training and/or multiple seeds before treating it as robust.

Entry 016 - Continued 8x8 Curriculum Checkpoint Training

- Date: 2026-04-29
- Change:
  - Continued training the `8x8`, `10` mines curriculum model across multiple saved checkpoints.
  - Saved checkpoints currently visible in `models/` include:
    - `models/best_cnn_deep_8x8_10m_curriculum_seed55.pt`
    - `models/best_cnn_deep_8x8_20m_curriculum_seed55.pt`
    - `models/best_cnn_deep_8x8_35m_curriculum_seed55.pt`
    - `models/best_cnn_deep_8x8_50m_curriculum_seed55.pt`
    - `models/best_cnn_deep_8x8_65m_curriculum_seed55.pt`
  - For resumed training, epsilon was raised again instead of continuing from the low checkpoint epsilon.
  - Learning rate was reduced to `0.0005` for fine-tuning.
- Hypothesis:
  - Resuming from a trained checkpoint with epsilon stuck near the minimum can limit exploration.
  - Raising epsilon again during continued training should help the model discover better actions on `8x8`.
  - Lower learning rate should reduce the risk of damaging useful weights while still allowing improvement.
- Example command:
  - ```
    python3 compare_curriculum_dqn.py --resume-checkpoint
    models/best_cnn_deep_8x8_50m_curriculum_seed55.pt --resume-epsilon-start 0.4
    --resume-learning-rate 0.0005 --resume-episodes 15000 --eval-games 1000
    --progress-every 500 --save-model-path
    models/best_cnn_deep_8x8_65m_curriculum_seed55.pt
    ```
- Result:
  - Loaded checkpoint: `models/best_cnn_deep_8x8_50m_curriculum_seed55.pt`
  - Continued for `15000` episodes
  - Reset training epsilon to `0.400`
  - Learning rate: `0.0005`
  - Saved checkpoint: `models/best_cnn_deep_8x8_65m_curriculum_seed55.pt`
  - Evaluation: win `36.70%`, reward `12.450`, steps `12.073`
- Outcome:
  - Continued 8x8 training is now producing meaningful win rates.
  - The agent survives much longer than early 8x8 runs, with average steps around `12`.
  - Next concern from UI inspection: the agent does not strongly prefer frontier cells near revealed information.

Entry 017 - Frontier Reward Bonus
    ```

- Date: 2026-04-29
- Change:
  - Added a controlled reward-shaping experiment.
  - Added `reward_mode="frontier"` that keeps current classic penalties unchanged:
    - invalid action outside board: `-5`
    - already revealed cell: `-2`
    - clicked mine / loss: `-10`
  - Keep base classic safe/win rewards:
    - safe move: `+1`
    - win step: `+21` total before any frontier bonus
  - Added a configurable bonus when the selected safe cell is a frontier cell.
  - Frontier action definition:
    - the selected cell is hidden before the move
    - it is adjacent to at least one revealed numbered cell
  - First bonus to test: `frontier_bonus=0.5`.
- Hypothesis:
  - The current classic reward gives the same `+1` for all safe cells.
  - It does not directly encourage choosing hidden cells near revealed information.
  - A small frontier bonus may teach the agent to prefer information-rich local moves without making frontier chasing dominate the win objective.
- Implementation:
  - Added `reward_mode="frontier"` to `MinesweeperEnv`.
  - Added `frontier_bonus` as a configurable environment parameter.
  - Before revealing a safe cell, the environment checks whether the clicked cell is adjacent to at least one revealed numbered cell.
  - If safe and frontier, the environment adds `frontier_bonus` to the safe reward.
  - Added CLI support for frontier reward in the training/evaluation scripts.
- Planned comparison:
  - Compare classic vs frontier reward on `8x8`, `10` mines.
  - Start from the same current checkpoint if doing continued training.
  - Use the same seed, same resume epsilon, same learning rate, same episode count, and same eval games.
- Result:
  - Not run yet.
- Outcome:
  - Implementation is ready.
  - Training/evaluation result is not run yet.

Entry 018 - Frontier Reward Fine-Tuning With Small Bonus and Small Learning Rate

- Date: 2026-04-30
- Change:
  - Continued from the best available `8x8`, `10` mines classic curriculum checkpoint.
  - Switched resumed training to `reward_mode=frontier`.
  - Used a smaller frontier bonus than initially planned:
    - `frontier_bonus=0.2`
  - Used a much smaller fine-tuning learning rate:
    - `resume_learning_rate=0.00005`
  - Raised epsilon again for continued exploration:
    - `resume_epsilon_start=0.4`
- Hypothesis:
  - The frontier reward should encourage moves near revealed numbered cells.
  - A smaller frontier bonus may be safer than `0.5` because it nudges behavior without overpowering the original win/loss objective.
  - A smaller learning rate may be better for fine-tuning because the checkpoint already has useful behavior.
- Parameters:
  - Board: `8x8`
  - Mines: `10`
  - Model: `CNN_DEEP DQN`
  - Replay: `uniform`
  - Starting checkpoint: `models/best_cnn_deep_8x8_65m_curriculum_seed55.pt`
  - Reward mode: `frontier`
  - Frontier bonus: `0.2`
  - Resume epsilon start: `0.4`
  - Resume learning rate: `0.00005`
  - Resume episodes: `15000`
  - Eval games: `1000`
  - Progress every: `500`
  - Seed: `55`
  - Saved checkpoint: `models/best_cnn_deep_8x8_80m_frontier_0.2_seed55.pt`
- Command:
  - `python3 compare_curriculum_dqn.py --resume-checkpoint models/best_cnn_deep_8x8_65m_curriculum_seed55.pt --resume-reward-mode frontier`

```
--frontier-bonus 0.2 --resume-epsilon-start 0.4 --resume-learning-rate 0.00005
--resume-episodes 15000 --eval-games 1000 --progress-every 500 --save-model-path
models/best_cnn_deep_8x8_80m_frontier_0.2_seed55.pt
```

- Progress snapshots:
  - Episode `500`: epsilon `0.312`, win `2.40%`, reward `-2.856`
  - Episode `3000`: epsilon `0.089`, win `15.80%`, reward `4.581`
  - Episode `6000`: epsilon `0.050`, win `27.40%`, reward `10.984`
  - Episode `8000`: epsilon `0.050`, win `31.20%`, reward `12.860`
  - Episode `12000`: epsilon `0.050`, win `32.20%`, reward `13.598`
  - Episode `15000`: epsilon `0.050`, win `30.60%`, reward `12.606`
- Final Result:
  - DQNAgent: win `47.60%`, reward `19.352`, steps `13.407`
- Delta vs previous documented `8x8_65m` checkpoint:
  - Previous: win `36.70%`, reward `12.450`, steps `12.073`
  - New: win `47.60%`, reward `19.352`, steps `13.407`
  - Win-rate delta: `+10.90` percentage points
  - Average-step delta: `+1.334`
- Outcome:
  - This is the best documented `8x8`, `10` mines result so far.
  - Frontier reward helped substantially in this focused seed-55 run.
  - A smaller frontier bonus and smaller learning rate may be better for fine-tuning than the initially planned `frontier_bonus=0.5` and `learning_rate=0.0005`.
  - From now on, report entries should include the exact parameters and full rerunnable command.

Entry 019 - Frontier Reward Fine-Tuning With `frontier_bonus=0.5`

- Date: 2026-04-30
- Change:
  - Repeated the frontier fine-tuning experiment from the same starting checkpoint as Entry 018.
  - Kept the smaller fine-tuning learning rate:
    - `resume_learning_rate=0.00005`
  - Changed only the frontier bonus:
    - Entry 018 used `frontier_bonus=0.2`
    - This run used `frontier_bonus=0.5`
- Hypothesis:
  - A larger frontier bonus may encourage the agent more strongly to click cells near revealed information.
  - Keeping the same checkpoint, learning rate, epsilon reset, episode count, and eval games makes this a controlled comparison against Entry 018.

- Parameters:
  - Board: `8x8`
  - Mines: `10`
  - Model: `CNN_DEEP DQN`
  - Replay: `uniform`
  - Starting checkpoint: `models/best_cnn_deep_8x8_65m_curriculum_seed55.pt`
  - Reward mode: `frontier`
  - Frontier bonus: `0.5`
  - Resume epsilon start: `0.4`
  - Resume learning rate: `0.00005`
  - Resume episodes: `15000`
  - Eval games: `1000`
  - Progress every: `500`
  - Seed: `55`
  - Saved checkpoint: `models/best_cnn_deep_8x8_80m_frontier_0.5_seed55.pt`
- Command:
  - `python3 compare_curriculum_dqn.py --resume-checkpoint models/best_cnn_deep_8x8_65m_curriculum_seed55.pt --resume-reward-mode frontier --frontier-bonus 0.5 --resume-epsilon-start 0.4 --resume-learning-rate 0.00005 --resume-episodes 15000 --eval-games 1000 --progress-every 500 --save-model-path models/best_cnn_deep_8x8_80m_frontier_0.5_seed55.pt`
- Progress snapshots:
  - Episode `500`: epsilon `0.312`, win `2.00%`, reward `-2.053`
  - Episode `3000`: epsilon `0.089`, win `13.00%`, reward `6.016`
  - Episode `4000`: epsilon `0.054`, win `25.80%`, reward `12.797`
  - Episode `7500`: epsilon `0.050`, win `33.60%`, reward `17.162`
  - Episode `10000`: epsilon `0.050`, win `33.60%`, reward `17.932`
  - Episode `14000`: epsilon `0.050`, win `33.60%`, reward `17.186`
  - Episode `15000`: epsilon `0.050`, win `32.40%`, reward `16.166`
- Final Result:
  - DQNAgent: win `49.10%`, reward `23.974`, steps `13.967`
- Delta vs Entry 018 (`frontier_bonus=0.2`):
  - Entry 018: win `47.60%`, reward `19.352`, steps `13.407`
  - Entry 019: win `49.10%`, reward `23.974`, steps `13.967`
  - Win-rate delta: `+1.50` percentage points
  - Average-step delta: `+0.560`
- Delta vs previous documented `8x8_65m` checkpoint:

- Previous: win `36.70%`, reward `12.450`, steps `12.073`
- New: win `49.10%`, reward `23.974`, steps `13.967`
- Win-rate delta: `+12.40` percentage points
- Average-step delta: `+1.894`
- Outcome:
- This is the best documented `8x8`, `10` mines result so far.
- With the smaller learning rate fixed at `0.00005`, `frontier_bonus=0.5` slightly beat `frontier_bonus=0.2` on win rate and average steps.
- Average reward is higher partly because the reward scale is larger with a bigger frontier bonus, so win rate and average steps are more reliable comparison metrics.

Template For Next Entries

Copy this block for each new improvement:

```
## Entry XXX - <Short title>
- Date: YYYY-MM-DD
- Change:
- Hypothesis:
- Parameters:
- Command:
- Progress snapshots:
- Final Result:
  - RandomAgent: win `..`, reward `..`, steps `..`
  - QLearningAgent: win `..`, reward `..`, steps `..`
  - DQNAgent: win `..`, reward `..`, steps `..`
- Delta vs RandomAgent:
- Delta vs QLearningAgent:
- Outcome:
```